

ESP32-S3 QR Pose Math Study Notebook

This notebook is the single primary study document for this project.

Goal: understand the full path from QR corners to stable pose output and robotics-ready IK input.

What you should be able to do after finishing this notebook:

1. Explain the camera and homography equations in plain language.
2. Trace how firmware turns detections into tracked output.
3. Tune robustness and speed with confidence, not guesswork.
4. Convert ESP32 /data output into a safe robotics update packet.

1. Coordinate Frames

We use these frames:

- Camera frame $\{C\}$
- QR frame $\{Q\}$ (origin at QR center, on QR plane)
- Robot base frame $\{B\}$ (for integration)

A point on QR plane:

$$P_Q = [X_Q, Y_Q, 0, 1]^T$$

2. Camera Projection Model

Pinhole camera model:

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} R & t \end{bmatrix} \begin{bmatrix} X_Q \\ Y_Q \\ 0 \\ 1 \end{bmatrix}$$

with intrinsic matrix

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

3. Homography for Planar QR

Because QR corners lie on one plane ($Z_Q=0$), mapping plane to image is:

$$\lambda p = H P_{plane}, P_{plane} = [X_Q, Y_Q, 1]^T$$

From 4 corner correspondences, solve for homography H (8 DoF up to scale).

4. Pose from Homography

Normalize homography:

$$\widetilde{H} = K^{-1} H = [h_1 h_2 h_3]$$

Then estimate rotation and translation:

$$r_1 = \frac{h_1}{\|h_1\|}, r_2 = \frac{h_2}{\|h_2\|}, r_3 = r_1 \times r_2$$

$$t = \frac{h_3}{s}$$

where s is a scale factor from homography column norms.

5. Euler Angles

One common extraction:

$$\text{roll} = \arctan 2(R_{32}, R_{33})$$

$$\text{pitch} = \arctan 2(-R_{31}, \sqrt{R_{11}^2 + R_{21}^2})$$

$$\text{yaw} = \arctan 2(R_{21}, R_{11})$$

In this project outputs are in degrees for readability and robotics integration.

6. Robustness Model (Tracking + Confidence)

When a raw observation is missing, the tracker can keep an estimated output for a short window.

Confidence decay model:

$$c(\Delta t) = \exp(-\Delta t / \tau)$$

Acceptance logic idea:

$$\text{accept} = (p \geq p_{\text{accept}})$$

Track validity logic idea:

$$\text{track_valid} = (\Delta t < T_{\text{max}}) \wedge (c > c_{\text{min}})$$

Practical interpretation:

1. Larger τ keeps confidence high longer (smoother but can become stale).
2. Smaller τ drops confidence faster (safer but less continuous).
3. Higher p_{accept} rejects noisy observations (cleaner but may miss updates).
4. Lower p_{accept} accepts more updates (responsive but can add jitter).

7. Latency vs Robustness Trade-Off

For this project profile, robustness in good light is the first priority and latency is secondary.

Desired operating goal in good light:

1. Detect all visible QR codes across the planned scenarios.
2. Decode all detected QR codes across the planned scenarios.

3. Spend more processing time if needed, as long as detection and decode robustness improve.

Measured acceptance floor during development:

1. detection_ratio >= 0.90
2. decode_ratio_when_detected >= 0.90

These 0.90 values are minimum working targets, not the ideal final goal. The ideal good-light goal is to push both metrics as close to 1.00 as the hardware allows.

Main tuning knobs (firmware):

- QR_DOWNSCALE
- QR_USE_ADAPTIVE_BINARIZE
- TRACK_MAX_HOLD_MS
- TRACK_DECAY_TAU_MS
- TRACK_MIN_CONF
- OBS_ACCEPT_PROB

Suggested tuning order for beginners:

1. Stabilize detection first (lighting, distance, focus, and preprocessing).
2. Stabilize decode second (multi-pass behavior and image quality).
3. Stabilize continuity third (tracker hold, decay, and accept threshold).
4. Optimize latency last, only if robustness targets stay high.

A practical latency budget view:

$$T_{total} = T_{capture} + T_{vision} + T_{tx} + T_{ik} + T_{actuation}$$

In this project, T_{vision} is usually the dominant term when robustness settings are aggressive. That is acceptable here if it improves detection and decode reliability.

7A. Practical Robust Mode Profiles

These are practical starting profiles for good-light, robustness-first testing on limited hardware.

Priority order for this project:

1. Detect all QR codes in the tested good-light scenarios.
2. Decode all QR codes in the tested good-light scenarios.
3. Reduce latency only after the first two goals are consistently satisfied.

Profile	QR_DOWNSCALE	QR_USE_ADAPTIVE_BINARIZE	QR_MAX_HOLD_MS	TRACK_DECAY_TAU_MS	TRACK_MIN_CONF	OBS_ACCEPT_PROB	Use Case	
Safe default	1	1	0	3500	2600	0.10	0.15	Current balanced robustness-

Profile	QR_DOWNSCALE	QR_USE_ADAPTIVE_BINA RIZE	QR_MAIN_USE_ADAPTIVE	TRACK_MAX_HOLD_MS	TRACK_DECAY_TAU_MS	TRACK_MIN_CONF	OBS_ACC_EPT_PROB	Use Case
								first baseline in firmware are
Good-light max robustness	1	1	0	5000	3500	0.08	0.12	Use when you are willing to spend more time to keep detections and decoded outputs alive in good light

How to use this table:

1. Start with Safe default.
2. Measure repeated runs in your good-light scenarios.
3. If detections are present but decode is inconsistent, move toward Good-light max robustness.
4. Only reduce compute cost after detection and decode are consistently strong.

Important warning: a stricter robustness profile can improve continuity, but it can also hide stale estimates if hold time becomes too long. Always check whether outputs are truly observed or only tracked.

8. Robotics Integration (Transform Chain)

To produce a target pose in robot base frame:

$${}^B T_E = {}^B T_C \cdot {}^C T_Q \cdot {}^Q T_E$$

Frame meaning:

- $\{B\}$: robot base frame

- $\{C\}$: camera frame
- $\{Q\}$: QR frame
- $\{E\}$: desired end-effector frame

Vision estimates ${}^C T_Q$. Calibration gives ${}^B T_C$. Task setup defines ${}^Q T_E$.

Pose matrix form used by robotics:

$${}^C T_Q = \begin{bmatrix} R(\phi, \theta, \psi) & t \\ 0^T & 1 \end{bmatrix}, t = [t_x, t_y, t_z]^T$$

Important rule: keep one Euler convention end-to-end (vision, middleware, and IK).

9. Function-by-Function Study Path (Speed + Robustness Gains)

You can improve this system significantly by studying each function in firmware and understanding its side effects.

Start here (in order):

1. process_qr_frame: overall scheduling, pass orchestration, publication.
2. run_quirc_scan + decode_quirc_payload: detection/decode behavior and ECC outcomes.
3. compute_qr_pose path (reorder, homography, decomposition, Euler): pose stability and sign consistency.
4. detection_probability + tracker functions: continuity, gating, and stale estimate behavior.
5. data_handler/status_handler + PC polling intervals: end-to-end latency and load behavior.

This is the fastest way to gain both robustness and speed without blind trial-and-error.

10. Failure Modes and Safeguards

Recommended safeguards for deployment:

1. No-detection burst: hold last valid pose and reduce robot speed.
2. Estimated-only for too long: freeze orientation updates first.
3. Confidence collapse: switch to safe search posture or pause motion.

These rules keep behavior predictable when vision quality drops suddenly.

11. Calibration Requirements (Recommended Order)

For real robot integration, calibration quality is as important as detection quality.

Recommended order:

1. Camera intrinsics (K and distortion model).
2. Camera-to-base extrinsics ${}^B T_C$.
3. Tool/task offset ${}^Q T_E$ for final approach behavior.

If these are wrong, even perfect QR detection can still produce wrong robot motion.

12. Filtering for Control Stability

To reduce jitter, apply first-order filtering on incoming target values:

$$x_k = \alpha z_k + (1 - \alpha) x_{k-1}$$

Use separate gains for translation and rotation.

Important detail: angles must be wrapped when blending near $\pm 180^\circ$ to avoid sudden jumps.

Practical hint:

1. Higher α gives faster response but more noise.
2. Lower α gives smoother motion but more lag.

13. Controller-Side Confidence Gating

A robust controller should gate visual updates before applying them to motion.

One practical gate:

$$\text{accept} = (c > c_{\min}) \wedge (e = 0 \text{ or } \Delta t < T_{\text{hold}})$$

Where:

- c : confidence from vision
- e : estimated flag (0 = observed, 1 = tracker-estimated)
- Δt : age of latest tracked estimate

Policy suggestion:

1. If accepted: update full target pose.
2. If rejected: hold last safe target or update with very low gain.
3. If estimated-only persists too long: freeze orientation first, then translation if needed.

14. IK Data Packet

Recommended fixed array:

```
pose_ik = [tx_mm, ty_mm, tz_mm, roll_deg, pitch_deg, yaw_deg, confidence,
           decoded_flag, estimated_flag]
```

Use confidence gating in control: for example, accept update only if confidence > 0.35.

15. Reusable Integration Pattern

This same downstream interface can be reused with other visual front ends (ArUco, AprilTag, keypoint trackers).

If packet format and units are stable, most IK/controller code can remain unchanged while only the vision front end is replaced.

Example: convert one /data QR record to IK packet

```
def qr_to_ik_packet(qr):
    return [
        float(qr.get('tx', 0.0)),
        float(qr.get('ty', 0.0)),
        float(qr.get('tz', 0.0)),
        float(qr.get('roll', 0.0)),
        float(qr.get('pitch', 0.0)),
        float(qr.get('yaw', 0.0)),
        float(qr.get('confidence', 0.0)),
```

```

    1 if bool(qr.get('decoded', False)) else 0,
    1 if bool(qr.get('estimated', False)) else 0,
]

```

```

example_qr = {
    'tx': -0.6, 'ty': -10.0, 'tz': 202.7,
    'roll': -10.6, 'pitch': 0.9, 'yaw': -2.5,
    'confidence': 1.0, 'decoded': True, 'estimated': False
}

```

```

ik_packet = qr_to_ik_packet(example_qr)
ik_packet

```

16. Study Checklist and Acceptance Criteria

Core study checklist:

1. Re-derive projection equation and explain each term.
2. Re-derive homography-based pose extraction by hand.
3. Validate Euler convention against firmware output.
4. Explain confidence decay and acceptance thresholds.
5. Build and test your IK packet parser from /data JSON.
6. Explain one safe fallback policy for low-confidence runs.

Acceptance criteria for this project (good-light robustness-first):

1. Minimum working threshold: `detection_ratio` ≥ 0.90
2. Minimum working threshold: `decode_ratio_when_detected` ≥ 0.90
3. Desired final good-light goal: push both metrics toward 1.00 across all planned scenarios
4. Treat latency as secondary unless it starts harming camera stability or overall usability

If below target, debug in this order:

1. Lighting quality, glare control, and marker size/distance.
2. Focus quality and motion blur reduction.
3. Preprocess and multi-pass settings.
4. Tracker acceptance and confidence thresholds.
5. Only after that, revisit lower-level pose or transport details.